

Data Preparation, Feature Engineering and Model Exploration

Reproducible temporal customer modeling for future-purchase inactivity

Course: Samsung Innovation Campus — AI in Marketing Capstone

Team: Group 7

Project: GrowthPilot AI

Prepared by: Şahin Başcı

Date: 9 September 2026

Status: Final local academic package

AI AUTHORSHIP DISCLOSURE — This document was drafted with OpenAI Codex/ChatGPT assistance and checked against repository evidence. Reported project metrics come from executed, versioned artifacts; external claims are cited. Human review remains required.

Part I — Data preparation and feature engineering

1. Overview and collection

The pipeline downloads UCI Online Retail II from its stable repository URL, verifies SHA-256 572e36277c2390fbfde10664750731e0a86f55e33470d91919085f0408e67bfb, profiles both worksheets and writes deterministic reports. Raw and customer-level prepared files are local/ignored; aggregate evidence and scripts are versioned.

2. Cleaning and validation

Step	Rows / rule	Rationale
Raw input	1,067,371	Preserved external-source evidence
Exact duplicate removal	34,335 removed	Avoid repeated identical transaction lines
Identity/date usability	235,151 excluded	Customer snapshots require stable ID and time
Normalized retained rows	797,885	Canonical names/types after deterministic rules
Eligible purchase event	Positive quantity and price; non-cancellation; pre-cutoff	Separate purchases from returns/cancellations

Evidence class: project-generated from artifacts/reports/data_preparation.json.

Missing customer identifiers are not imputed because arbitrary identity assignment would create false histories. Missing descriptions do not block customer-level behavior features. Nonpositive

quantity/price and cancellation invoices are excluded from eligible purchase events but remain part of source-quality evidence. Monetary outliers are not winsorized before aggregation; log1p and standardized transformations reduce scale dominance inside fitted pipelines.

3. Temporal snapshots and leakage prevention

Split	Cutoff(s)	Rows	Inactivity rate
Training	2010-06-01 monthly through 2010-12-01	20,677	0.483823
Validation	2011-03-01	3,349	0.577187
Calibration	2011-06-01	2,659	0.499436
Final test	2011-09-01	2,772	Sealed until final evaluation

All features use events strictly before the cutoff; each 90-day label window is fully observed.

4. EDA

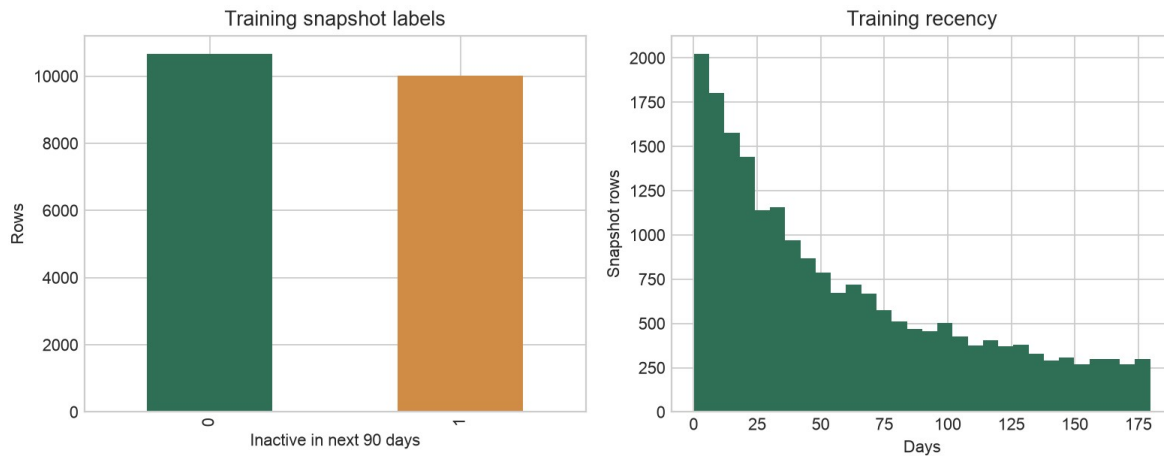


Figure 1. Label prevalence and recency across training snapshots.

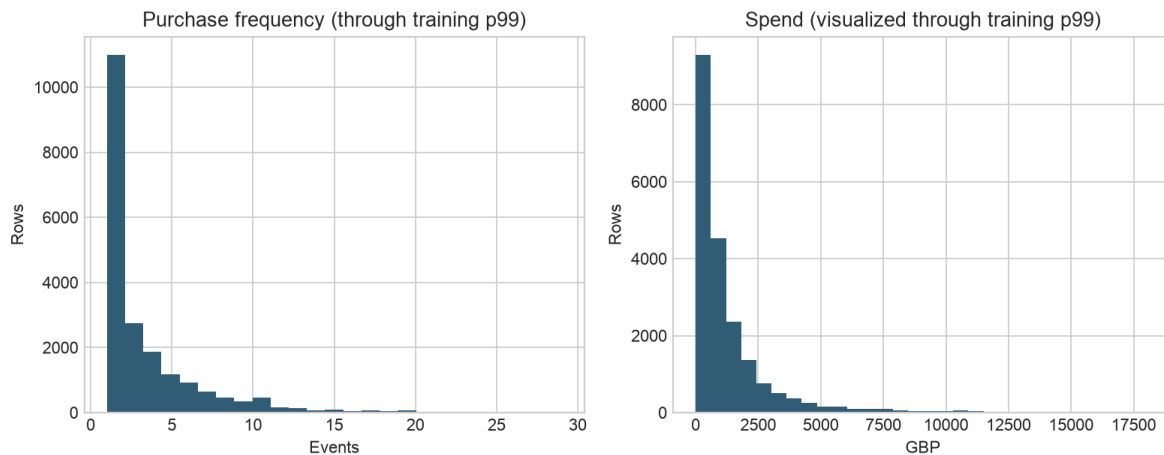


Figure 2. Frequency and spend distributions support transformed modeling.

Training contains 20,677 customer snapshots with 48.3823% inactivity labels. Validation has 3,349 rows with 57.7187% inactivity. The prevalence shift is one reason accuracy is unsuitable as the primary measure. Skew and repeat observations also motivate time-based evaluation and customer-behavior aggregates rather than random line-level splitting.

5. Feature design

Feature family	Examples	Rationale
Recency/tenure	recency_days, tenure_days	How recently and how long the relationship has existed
Frequency/cadence	invoice_count, active_days, mean/median/max gap	Repeat behavior and purchase periodicity
Monetary	gross/net spend, average order value, return value	Commercial magnitude and reversal behavior
Product breadth	distinct products, quantity, basket breadth	Depth and diversity of engagement
Windowed behavior	30/60/90-day counts/spend; trend deltas	Recent acceleration or decline before cutoff
Context	country and snapshot month	Market/time context, encoded inside the pipeline

6. Scaling, normalization and encoding

The scikit-learn ColumnTransformer fits preprocessing only on training data. Numeric columns use median imputation, log1p where defined and StandardScaler for logistic regression. Categorical values use most-frequent imputation and one-hot encoding with unknown-category tolerance. Tree candidates use compatible encoded input without assuming that scaling improves trees.

```
numeric = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('scale', StandardScaler()),
])
categorical = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore')),
])
model = Pipeline([('preprocess', ColumnTransformer(...)),
                  ('classifier', LogisticRegression(C=0.01, max_iter=2000))])
```

Part II — Model exploration

1. Candidate selection rationale

The recency heuristic is a business baseline; the dummy prior detects whether a model adds ranking information. Logistic regression is transparent, fast and regularizable. Random forest captures nonlinear interactions with modest preprocessing. LightGBM is an efficient boosted-tree challenger. The concept deck anticipated LightGBM, but the experiment did not privilege it: validation PR-AUC selected logistic regression.

Candidate	Strength	Weakness
Recency heuristic	Simple, explainable business reference	Ignores frequency, value and cadence
Dummy prior	Calibration/prevalence sanity check	No individualized ranking
Logistic regression	Transparent, efficient, stable regularization	Linear log-odds boundary; encoded interactions limited
Random forest	Nonlinear interactions and robustness	Larger model; probability calibration may degrade
LightGBM	Strong nonlinear tabular learner	More tuning/interpretation complexity and overfit risk

2. Training, hyperparameters and validation

Candidate preprocessing and models are fit on seven pre-test training cutoffs. Model family is selected by the single 2011-03-01 validation snapshot. A later 2011-06-01 calibration snapshot is reserved for sigmoid probability calibration and the operational threshold; the final 2011-09-01 cutoff is never used during exploration.

Model	Key parameters	Validation PR-AUC	ROC-AUC	Brier	Top-10% lift
Recency	clip(recency/180)	0.715529	0.689417	0.248166	1.292940
Dummy	prior	0.577187	0.500000	0.244072	0.961948
Logistic	C=1.0	0.805322	0.782819	0.213300	1.510154
Random forest	300 trees; leaf=10	0.803517	0.782053	0.195463	1.520498
LightGBM	300 trees; lr=.05; leaves=31	0.805223	0.776941	0.200875	1.546357

Selection metric: validation PR-AUC. Initial winner: logistic regression.

3. Evaluation interpretation

Logistic PR-AUC 0.805322 narrowly exceeds LightGBM 0.805223 and random forest 0.803517 in initial exploration. Tree candidates have better uncalibrated Brier values, which motivates a separate calibration step rather than changing the selection metric after seeing results. The recency baseline is meaningfully weaker but nontrivial, confirming that simple recency carries signal.

4. Reproducibility

```
make data-download
make data-profile
make data-prepare
make features-build
make train
# Outputs include split hashes, MLflow run IDs, parameters and validation metrics.
```

Every split has a SHA-256 checksum and versioned target/feature/split identifiers. MLflow records model runs. The final test flag in model_exploration.json is false, demonstrating that exploration completed before final-test access.

5. Conclusion

The preparation pipeline is leakage-aware, deterministic and suitable for the chosen historical study. Initial exploration selected logistic regression for refinement. This conclusion is limited to the stated validation cutoff and metric; it does not claim universal superiority or production readiness.

References

Chen, D. (2012). Online Retail II [Dataset]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5CG6D>

scikit-learn developers. Model evaluation, preprocessing and probability calibration documentation.
<https://scikit-learn.org/stable/>

Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30.

Project evidence: artifacts/reports/dataset_profile.json, artifacts/reports/data_preparation.json, artifacts/eda, artifacts/ml/model_exploration.json and the corresponding scripts/tests.